

1. Diseño de la interfaz remota.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface IDNSService extends Remote {
    // Sustituye a tu comando "QUERY"
    String query(String nombre) throws RemoteException;

    void add(String nombre, String ip, int ttl) throws RemoteException;
}
```

- La interfaz:
El cliente no necesita saber *cómo* está implementado el servidor (si usa un HashMap, una base de datos o si borra los datos con un hilo), solo necesita conocer esta interfaz para saber *qué* métodos puede llamar.
- Herencia de Remote:
Le indica a la Máquina Virtual de Java (JVM) que los métodos de esta interfaz pueden ser invocados desde una máquina distinta a la que reside el objeto. Sin esto, RMI no permitiría la conexión distribuida que habéis hecho.
- El uso de RemoteException:
Esto es una diferencia crítica respecto a los métodos locales. En un programa normal, un query nunca fallaría por "problemas de red". En sistemas distribuidos, tienes que obligar al programador a gestionar fallos de red (caídas, latencia, timeouts) usando bloques try-catch.
- Paso de parámetros y Tipado fuerte:
Aquí usas tipos nativos (String, int). Java se encarga de la serialización (convertir esos objetos a bytes y viceversa) de forma transparente. Esto elimina errores de formato que eran comunes en la práctica de Python.

2. Decisiones adoptadas.

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.concurrent.ConcurrentHashMap;

public class DNSServiceImpl extends UnicastRemoteObject implements IDNSService {

    private ConcurrentHashMap<String, Registro> cache;

    class Registro {
        String ip;
        long expiracion;

        Registro(String ip, int ttl) {
            this.ip = ip;
            this.expiracion = System.currentTimeMillis() + (ttl * 1000L);
        }
    }
}
```

- Unicast Remote: Esto es lo que permite que el objeto "viva" en el servidor y pueda escuchar peticiones remotas. Al heredar de aquí, Java se encarga automáticamente de crear los *stubs* y gestionar la comunicación por red sin que tú tengas que programar los sockets manualmente.
- ConcurrentHashMap: RMI es multihilo (puede atender a varios clientes a la vez). Si dos personas intentan añadir un dominio al mismo tiempo y usáis un mapa normal, el servidor podría "petar" o dar errores de memoria. El ConcurrentHashMap permite que varios hilos lean y escriban de forma segura sin bloquearse.
- Registro: En lugar de tener dos mapas separados (uno para IPs y otro para tiempos), guardáis todo en un solo objeto Registro. Esto mantiene el código mucho más limpio y organizado.

```
@Override
public String query(String nombre) throws RemoteException {
    Registro reg = cache.get(nombre);

    if (reg == null) return "NOTFOUND";

    if (System.currentTimeMillis() > reg.expiracion) {
        cache.remove(nombre);
        return "NOTFOUND (Expired)";
    }

    long ttlRestante = (reg.expiracion - System.currentTimeMillis()) / 1000;
    return "IP: " + reg.ip + " | TTL Restante: " + ttlRestante + "s";
}

@Override
public void add(String nombre, String ip, int ttl) throws RemoteException {
    cache.put(nombre, new Registro(ip, ttl));
    System.out.println("Añadido: " + nombre + " -> " + ip + " (TTL: " + ttl + ")");
}
}
```

- Gestión inteligente del TTL: En lugar de tener un reloj interno que borre datos cada segundo (que gastaría mucha batería/procesador), el servidor solo comprueba si un dato ha caducado cuando alguien pregunta por él.
- Transparencia en el manejo de datos: Aquí se ve la potencia de RMI. No habéis tenido que trocear el texto (como hacíais en Python con los Sockets). Simplemente recibís los objetos String e int directamente y los guardáis en el mapa. Java se ha encargado de todo el "trabajo sucio" de mover esos datos por la red.
- Retroalimentación en el servidor: Aunque la acción ocurre de forma remota (desde el PC de tu colega), el servidor imprime por pantalla lo que está pasando. Esto es vital en sistemas distribuidos para que el administrador sepa que el sistema está vivo y recibiendo peticiones correctamente.
- Cálculo del tiempo restante: Esto mejora la experiencia del usuario. Al restar la expiración menos el tiempo actual, el cliente no solo recibe la IP, sino que sabe exactamente cuántos segundos de vida le quedan a esa información en la caché.

```

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class ServidorDNS {
    public static void main(String[] args) {
        try {
            System.setProperty("java.rmi.server.hostname", "172.23.203.161");

            LocateRegistry.createRegistry(1099);

            DNSServiceImpl objetoRemoto = new DNSServiceImpl();

            Naming.rebind("ServicioDNS", objetoRemoto);

            System.out.println("Servidor DNS RMI funcionando correctamente...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

- Identidad de red: Usamos `System.setProperty` para asignar la IP real del servidor. Sin esto, el cliente no sabría a dónde enviar las peticiones fuera de localhost.
- El registro: Levantamos el Registry en el puerto 1099. Funciona como un listín telefónico donde el cliente busca el nombre del servicio para obtener su dirección.
- Publicación: Con `Naming.rebind`, registramos nuestra lógica bajo el nombre "ServicioDNS". Usamos `rebind` (sobrescribir) para que, si el servidor se reinicia, el registro se actualice sin errores.
- Resiliencia: Todo el arranque está protegido por un bloque try-catch, ya que cualquier fallo en la red o puerto ocupado impediría el inicio del servicio.

3. Reflexión técnica.

A. Del protocolo a los métodos:

- Sockets: La comunicación era manual. Había que diseñar tramas de texto (ej. "ADD:dominio:ip") y parsearlas con `split(":")`.
- RMI: La red es invisible. Invocamos métodos directamente (`objeto.add(...)`). El programador se centra en la lógica, no en el transporte.

B. Problemas resueltos:

- Adiós al Marshalling manual: Java empaqueta y envía los objetos automáticamente; ya no hay que convertir datos a bytes o strings a mano.
- Multihilo automático: RMI gestiona su propio "pool" de hilos. Puede atender a varios clientes a la vez sin que nosotros tengamos que crear Threads manualmente.

C. Nuevos desafíos:

- Latencia y Bloqueo: Al ser llamadas remotas, si la red falla o es lenta, el programa se detiene (bloquea) esperando respuesta, algo que no pasa en llamadas locales.
- Robustez obligatoria: El uso de RemoteException nos obliga a gestionar fallos de red en cada llamada mediante bloques try-catch.
- Configuración Crítica: RMI es más sensible que los sockets. Problemas como el hostname o cortafuegos (Firewalls) complican el despliegue en redes reales.

4. Instrucciones de ejecución.

- Paso 1: Compilación de todos los módulos Desde la terminal, en la carpeta raíz del proyecto, ejecutamos la compilación de todas las clases Java: `javac IDNSService.java DNSServiceImpl.java ServidorDNS.java ClienteDNS.java`
- Paso 2: Lanzamiento del Servidor:
 1. Es necesario abrir el archivo ServidorDNS.java y actualizar la dirección IP en la línea `java.rmi.server.hostname` con la IP actual de la máquina (ej. la de la red Wi-Fi o Hotspot).
 2. Ejecutamos el servidor: `java ServidorDNS`
 3. El terminal mostrará el mensaje: *"Servidor DNS RMI funcionando correctamente..."*.
- Paso 3: Lanzamiento del Cliente:
 1. En el PC cliente, nos aseguramos de que el archivo ClienteDNS.java apunte a la IP donde se está ejecutando el servidor.
 2. Ejecutamos el cliente: `java ClienteDNS`